

PowerShell Basics

PowerShell is Windows' scripting language and shell — built on .NET, object-oriented (unlike Bash which is text-based), and absolutely essential for Windows pentesting, AD attacks, and Windows SOC/log analysis. Since you're building toward AD-focused work, this is one of the highest-leverage things you can learn well.

1. PowerShell vs. Bash — The Core Difference

This is the single most important conceptual shift coming from Bash:

	Bash	PowerShell
Data passed between commands	Plain text	Objects (structured .NET objects)
Piping	Text stream, needs grep/awk/sed to parse	Pipe objects directly, filter by property
Naming	Inconsistent (<code>ls</code> , <code>grep</code> , <code>awk</code>)	Consistent Verb-Noun pattern

Example of the difference — getting running processes:

```
# Bash – text output, need to parse with awk/grep
ps aux | grep chrome | awk '{print $2}'
```

```
# PowerShell – returns actual Process objects, filter by property directly
Get-Process | Where-Object {$_.ProcessName -eq "chrome"} |
Select-Object Id
```

No text-parsing gymnastics needed — you're working with real objects that have properties.

2. Cmdlet Naming Convention — Verb-Noun

Every PowerShell command follows **Verb-Noun** format:

```

Get-Process          # retrieve something
Set-Location         # change something
New-Item             # create something
Remove-Item          # delete something
Start-Service        # start something
Stop-Process         # stop something

```

Common verbs:

Verb	Purpose
Get	Retrieve data
Set	Modify/configure
New	Create
Remove	Delete
Start / Stop	Control execution
Invoke	Run/execute something
Test	Check a condition (returns True/False)

```

Get-Command -Verb Get          # list all "Get-*" cmdlets available
Get-Help Get-Process           # built-in help for any cmdlet
Get-Help Get-Process -Examples # just usage examples

```

3. Variables

```

$name = "Omkar"
Write-Host "Hello, $name"

$count = 10
$count += 5          # $count is now 15

# Arrays
$ports = @(22, 80, 443, 3389)
Write-Host $ports[0]    # 22

```

```
# Hashtables (like dictionaries)
$user = @{Name="Omkar"; Role="Analyst"}
Write-Host $user.Name           # Omkar
```

`Write-Host` prints directly to console. `Write-Output` sends objects down the pipeline (important distinction — `Write-Host` output *can't* be piped/captured, `Write-Output` can).

4. The Pipeline — Objects, Not Text

```
Get-Process | Sort-Object CPU -Descending | Select-Object -
First 5
```

This chain: gets all processes → sorts by CPU usage (descending) → shows top 5. Each `|` passes full objects forward, not text.

```
Get-Service | Where-Object {$_.Status -eq "Running"}
```

`$_` represents "the current object in the pipeline" — equivalent conceptually to `awk`'s implicit field variables, but here it's a full object with properties, not a text field.

Inspecting what properties an object has:

```
Get-Process | Get-Member
```

This is one of the most useful commands to know — it shows every property/method available on whatever object you're working with, so you always know what you can filter/select on.

5. Conditionals

```
$target = "192.168.1.1"

if (Test-Connection -ComputerName $target -Count 1 -Quiet)
{
    Write-Host "$target is UP"
}
```

```
} else {  
    Write-Host "$target is DOWN"  
}
```

Comparison operators (different from Bash!):

```
-eq      # equal  
-ne      # not equal  
-gt      # greater than  
-lt      # less than  
-ge      # greater or equal  
-le      # less or equal  
-like    # wildcard string match  
-match   # regex match
```

```
if ($count -gt 10) { Write-Host "More than 10" }  
if ($name -like "0m*") { Write-Host "Starts with 0m" }  
if ($line -match "^\d+\.\d+\.\d+\.\d+$") { Write-Host "Looks like an IP" }
```

6. Loops

```
# foreach loop  
$ips = @("192.168.1.1", "192.168.1.2", "192.168.1.3")  
foreach ($ip in $ips) {  
    Write-Host "Pinging $ip"  
    Test-Connection -ComputerName $ip -Count 1  
}  
  
# for loop  
for ($i = 1; $i -le 254; $i++) {  
    Write-Host "192.168.1.$i"  
}  
  
# while loop  
$i = 0
```

```
while ($i -lt 5) {
    Write-Host "Count: $i"
    $i++
}

# ForEach-Object (pipeline version – very common)
1..5 | ForEach-Object { Write-Host "Number: $_" }
```

7. Functions

```
function Check-Port {
    param(
        [string]$Host,
        [int]$Port
    )
    $result = Test-NetConnection -ComputerName $Host -Port
$Port
    if ($result.TcpTestSucceeded) {
        Write-Host "[OPEN] $Host`:$Port"
    } else {
        Write-Host "[CLOSED] $Host`:$Port"
    }
}

Check-Port -Host "192.168.1.1" -Port 22
Check-Port -Host "192.168.1.1" -Port 445
```

The backtick ``` is PowerShell's escape character — used above so `:` doesn't get misread as part of variable syntax.

8. File & Directory Operations

Get-ChildItem C:\Users\Omkar\Documents	# like 'ls'
Get-ChildItem -Recurse -Filter *.txt	# recursive se
arch for .txt files	
Get-Content file.txt	# like 'cat'
Set-Content file.txt "new content"	# overwrite f

```

ile
Add-Content file.txt "appended line"           # append to f
ile
Copy-Item source.txt dest.txt
Move-Item old.txt new.txt
Remove-Item file.txt
Test-Path C:\Users\Omkar\file.txt              # check if fi
le/path exists (True/False)
New-Item -ItemType Directory -Path C:\NewFolder

```

Common aliases (for Bash muscle memory):

```

ls      → Get-ChildItem
cat     → Get-Content
cp      → Copy-Item
mv      → Move-Item
rm      → Remove-Item
pwd     → Get-Location
cd      → Set-Location
ps      → Get-Process
kill    → Stop-Process

```

These aliases exist specifically to ease the Linux-to-Windows transition, but the real cmdlet names are what you should learn to actually use effectively (aliases don't work the same way in scripts meant to run on other systems, and knowing full names helps you understand `Get-Help` output).

9. Practical Example — Simple Port Scanner

```

$target = "192.168.1.10"
$ports = @(21, 22, 23, 25, 80, 443, 3389, 445, 3306)

Write-Host "Scanning $target..."
foreach ($port in $ports) {
    $result = Test-NetConnection -ComputerName $target -Port $port -WarningAction SilentlyContinue
    if ($result.TcpTestSucceeded) {
        Write-Host "[OPEN] Port $port"
    }
}

```

```
}  
}  
Write-Host "Scan complete."
```

Run it: `powershell -File portscan.ps1`

10. Working with CSV/JSON (Common in Log/Report Parsing)

```
# Export process list to CSV  
Get-Process | Export-Csv -Path processes.csv -NoTypeInfo  
tion  
  
# Import and filter a CSV  
Import-Csv users.csv | Where-Object {$_.Department -eq "I  
T"}  
  
# Working with JSON (e.g., API responses – relevant to your  
SmartScan-style projects)  
$json = Get-Content data.json | ConvertFrom-Json  
Write-Host $json.name  
  
$obj = @{Name="Omkar"; Role="Analyst"}  
$obj | ConvertTo-Json
```

11. Execution Policy (Important Windows-Specific Concept)

Windows restricts script execution by default — this trips up almost everyone the first time:

```
Get-ExecutionPolicy
```

Policy	Meaning
<code>Restricted</code>	No scripts allowed at all (default on client Windows)
<code>RemoteSigned</code>	Local scripts run freely, downloaded scripts need a digital signature

Policy	Meaning
AllSigned	All scripts must be signed
Unrestricted	Everything runs, with a warning for downloaded scripts
Bypass	Nothing blocked, no warnings — often used for automation/testing

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

From a pentesting angle: this is exactly why attackers use `-ExecutionPolicy Bypass` when running scripts, or avoid touching disk entirely with in-memory execution — worth knowing as a concept even before you get deep into offensive PowerShell.

12. Remote Execution — PowerShell Remoting

This is a big one for both sysadmin work and lateral movement in AD attacks.

```
# Enable remoting (on target, needs admin)
Enable-PSRemoting -Force

# Run a single command on a remote machine
Invoke-Command -ComputerName Server01 -ScriptBlock { Get-Process }

# Open an interactive remote session
Enter-PSSession -ComputerName Server01

# Run against multiple machines at once
Invoke-Command -ComputerName Server01,Server02 -ScriptBlock { Get-Service } -Credential $cred
```

This uses **WinRM (port 5985/5986)** under the hood — this exact protocol/port is something you'll check for during AD enumeration (`nmap -p 5985`), since open WinRM + valid credentials is a direct lateral movement path (also what tools like **Evil-WinRM** use).

13. PowerShell in a Security Context (Study-Level Awareness)

Since PowerShell is deeply embedded in modern Windows attacks, a few concepts worth knowing at a study level:

a) Living off the land

PowerShell is installed by default on every modern Windows machine, and it's a legitimate admin tool — so attackers use it heavily because it doesn't require dropping obviously malicious binaries. This is why PowerShell logging is such a big deal defensively.

b) PowerShell logging (blue-team side, relevant to your SOC interest)

```
# Check if script block logging is enabled
Get-ItemProperty "HKLM:\SOFTWARE\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging"
```

Windows Event IDs to know:

- **4104** — Script Block Logging (captures actual script content executed — huge for detection)
- **4103** — Module Logging
- **400/403** — Engine start/stop

As a future SOC analyst, these Event IDs are what you'd query in a SIEM to catch obfuscated/malicious PowerShell activity.

c) AMSI (Antimalware Scan Interface)

Windows Defender (and other AV) hooks into PowerShell via AMSI to scan script content before execution — this is why attackers use obfuscation or "AMSI bypass" techniques, a well-documented cat-and-mouse area worth reading about once you're comfortable with the basics here.

Quick Reference Cheat Sheet

```
Get-Help <cmdlet> -Examples      # help for any command
Get-Process                      # list processes
Get-Service                      # list services
Get-ChildItem -Recurse           # recursive directory
listing
Get-Content file.txt             # read file
Test-NetConnection -ComputerName x -Port 445 # test a por
```

```
t
Test-Path C:\file.txt                # check if path exists
Where-Object {$_.Property -eq "x"}   # filter pipeline objects
Select-Object -First 5                # limit output
Get-Member                           # inspect object properties/methods
Invoke-Command -ComputerName x -ScriptBlock {} # remote execution
Get-ExecutionPolicy / Set-ExecutionPolicy # script execution rules
whoami /priv                          # (cmd, but often run from PS too)
```